

Navigation

Navigator 1.0

Navigator 1.0 is the traditional way of handling navigation where:

- Screens are called **routes**
 - Navigation works like a **stack (LIFO)**
 - You **push** a new screen → it goes on top
 - You **pop** → go back to previous screen
-

Key Concept (Stack)

Screen A → Screen B → Screen C
 ↑ ↑ ↑
bottom middle top

- `push()` → add screen
 - `pop()` → remove screen
-

Example (2 Screens)

Main File

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: ScreenOne(),
    );
  }
}
```

Screen One

```
class ScreenOne extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Screen One")),
    );
  }
}
```

```

        body: Center(
          child: ElevatedButton(
            child: Text("Go to Screen Two"),
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => ScreenTwo(),
                ),
              );
            },
          ),
        ),
      ),
    );
  }
}

```

Screen Two

```

class ScreenTwo extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Screen Two")),
      body: Center(
        child: ElevatedButton(
          child: Text("Go Back"),
          onPressed: () {
            Navigator.pop(context);
          },
        ),
      ),
    );
  }
}

```

1. Navigator.push()

Used to **open a new screen** (adds to stack)

Concept:

- Current screen stays in memory
- New screen comes on top

✅ Example:

```

Navigator.push(
  context,
  MaterialPageRoute(

```

```
        builder: (context) => SecondScreen(),
      ),
    );
```

Flow:

Screen1 → Screen2

2. Navigator.pop()

Used to **go back to previous screen**

Concept:

- Removes the top screen from stack

 Example:

```
Navigator.pop(context);
```

Flow:

Screen1 ← Screen2 (removed)

3. Navigator.pushReplacement()

Used to **replace current screen with new one**

Concept:

- Current screen is removed
- New screen is added

Example:

```
Navigator.pushReplacement(
  context,
  MaterialPageRoute(
    builder: (context) => HomeScreen(),
  ),
);
```

Flow:

Login → Home
(Login removed)

Common use:

- After login (user should NOT go back)
-

4. Navigator.pushAndRemoveUntil()

Used to **clear stack and open new screen**

Concept:

- Removes multiple (or all) previous screens
- Opens new screen

Example:

```
Navigator.pushAndRemoveUntil(  
  context,  
  MaterialPageRoute(  
    builder: (context) => DashboardScreen(),  
  ),  
  (route) => false,  
);
```

Flow:

Login → OTP → Profile → Dashboard

↓

Dashboard only (all removed)

Named Routing

◆ 1. What is Named Routing?

In Flutter SDK, instead of writing:

```
Navigator.push(  
  context,  
  MaterialPageRoute(builder: (context) => DetailsScreen()),  
);
```

You use a **route name** like:

```
Navigator.pushNamed(context, '/details');
```

Here '/details' is a **named route**

◆ 2. How Named Routes Work (Core Idea)

Flutter maintains a **map (dictionary) of route names → widgets**

```
MaterialApp(  
  routes: {  
    '/': (context) => HomeScreen(),  
    '/details': (context) => DetailsScreen(),  
  },  
);
```

Internal Concept

```
'/details' → DetailsScreen()  
'/profile' → ProfileScreen()
```

When you call:

```
Navigator.pushNamed(context, '/details');
```

Flutter looks up the map and loads that screen

◆ 3. Basic Example (Step-by-Step)

Step 1: Define Routes

```
void main() {  
  runApp(  
    MaterialApp(  
      initialRoute: '/',  
      routes: {  
        '/': (context) => HomeScreen(),  
        '/second': (context) => SecondScreen(),  
      },  
    ),  
  );  
}
```

Step 2: Navigate

```
Navigator.pushNamed(context, '/second');
```

Step 3: Go Back

```
Navigator.pop(context);
```

◆ 4. Passing Data in Named Routes

Named routes don't directly accept constructor parameters like normal push. Instead, you pass **arguments**.

Send Data

```
Navigator.pushNamed(  
  context,  
  '/details',  
  arguments: 42,  
);
```

Receive Data

Inside the screen:

```
final args = ModalRoute.of(context)?.settings.arguments as int;
```

Problem

This approach is:

- Not type-safe
- Can get messy for complex data